

Spatial-Corridor and Time-Window Costs for Robust Robotic Tennis Hitting

Abstract—Robotic tennis hitting needs swings valid under ball-prediction errors, yet point targets compress the solution into a single, prediction-sensitive point. From MPC-iLQR’s point-target terminal, we study two switches: a spatial corridor (hinge position-deviation running cost along the predicted ball trajectory, racket radius as half-width) and a candidate-window terminal replacing the single-point target by a differentiable softmin over candidate states along that trajectory. On a dual-arm RM-65B in simulation (110k episodes, same-seed pairing), a four-way ablation, a perturbation grid, and a speed-limit pairing test them: the window carries nearly all nominal gain over the point target (+17.4 pp at 9 m/s, +32.7 pp at 12 m/s); under a target-point bias with unbiased trajectory geometry the corridor alone is significant (+4.4/+6.9 pp at 9/12 m/s), and once the window is active no further corridor gain is detected. Under the nominal 1.0 m/s TCP limit the soft-aggregation tiers fall below the point target (−11.9 to −12.7 pp); the corridor-only tier gains +5.8 pp. Each layer’s regime of benefit and failure is delimited; the cost structure should follow the speed budget.

Index Terms—model predictive control, iterative linear-quadratic regulator, trajectory optimization, ball hitting, uncertainty-aware planning

I. INTRODUCTION

In hitting, the swing must be committed in advance of contact on the strength of a ball prediction. Figure 1 contrasts the point-target terminal – a single time–space pair – with the two cost switches we study: a spatial corridor added as a running cost, and a candidate window that replaces the single-point terminal. Any plan rests on a ball prediction, and prediction error is unavoidable (sensing noise, model mismatch). Its cost is asymmetric: on our perturbation grid a 0.2 m hit-point offset alone lowers the full-system hit rate by 21.5 pp, a 100 ms timing shift by 4.1 pp (both at 9 m/s, the tested maxima).

The limits of the point-target paradigm. A common formulation in trajectory optimization casts hitting as a *point-target* problem [1], [2]: a terminal cost penalizes the end-effector for deviating, at a prescribed instant t_{hit} , from a single predicted target, which a prediction error shifts away from the true contact state. Practical alternatives rely on high-bandwidth sensing or on the implicit robustness of large-scale learning, but neither answers *how the cost function itself should be built to stay robust under prediction error*.

The gap. A feasible hitting solution is essentially a *spatiotemporal feasible set*: the racket face need only sweep, with an appropriate pose, through a spatial region along the ball trajectory at some candidate instant. The point target compresses this set into a single point. Non-point-target robust planners exist [3], [4], but we are not aware of prior work that

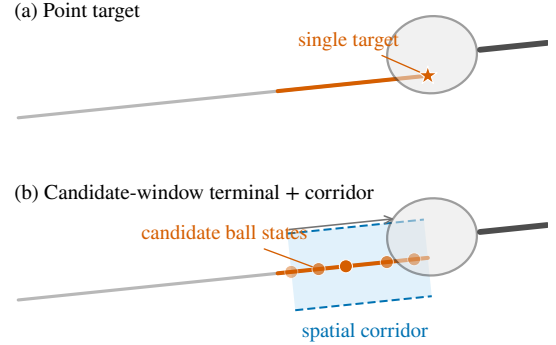


Fig. 1. Top-view schematic of the two cost switches. The spatial corridor penalizes deviations outside the blue band; the candidate-window terminal replaces the single target by candidate states along the trajectory (dots: an illustrative subset, all evaluated at the same terminal state). Panels share coordinates and scale (Figure 3).

attributes temporal and spatial robustness quantitatively under controlled perturbations. We therefore ask: (i) does relaxing the point target improve the hit rate? (ii) are the candidate-set and corridor gains complementary? (iii) does the relaxation survive a constrained execution speed?

Method and contributions. Starting from the point-target terminal of MPC-iLQR, this paper studies the two-switch cost family (5): a *spatial corridor* adds a hinge position-deviation running cost along the predicted ball trajectory, while a *candidate-window softmin terminal* can replace the single-point terminal by a soft aggregation over candidate ball states sampled along that trajectory.

- **C1 (method):** a trajectory-induced candidate-window terminal: candidate racket states sampled from the predicted ball trajectory, aggregated by a differentiable softmin into MPC-iLQR — one shared terminal state, a target set induced by timing uncertainty rather than contact-time selection;
- **C2 (method):** a geometric-tolerance corridor regularizer: a running cost from the incoming-ball direction and the racket radius, penalizing only perpendicular deviations outside the corridor;
- **C3 (analysis):** controlled same-seed evaluation (four-tier ablation, perturbation grid, TCP speed-limit pairing) identifying when each mechanism helps, adds nothing detectable, or hurts.

Artifact. An anonymized artifact containing code, configurations, per-episode results, and reproduction scripts is available at https://anonymous.4open.science/r/Tennis_robot_789741.

Section II–Section VII review related work, formalize the problem, present the method, report experiments and results, and conclude.

II. RELATED WORK

A. Hitting and Throwing Tasks

High-speed hitting spans kinematic planning and trajectory optimization: polynomial-primitive batting under 1 kHz visual servoing [5], biomimetic state-machine racket driving [6], and learning-based variants that select and generalize striking movements [7]. Recent systems [8]–[10] hit with fixed-horizon MPC, hierarchical planning with learned whole-body control, and motion primitives with MPC and Bayesian optimization; catching likewise hinges on predicting the object state under uncertainty and re-timing the reach [11]. These systems commonly combine point-target objectives with high-bandwidth perception or large training sets. This paper asks the complementary question under *more constrained* conditions: how much robustness the cost design itself can buy.

B. Trajectory Optimization and Tube MPC

iLQR/DDP [1], [2], [12] are the standard solvers for dynamic manipulation; tube MPC theory [13] handles feasibility under disturbances via state-tube constraints, and Tube Acceleration [14] introduces a reachable tube for release-time robustness in throwing. Our corridor migrates the tube concept to the *task-space path* of hitting, but as a soft cost rather than an invariance-based tube: we claim neither disturbance invariance nor recursive feasibility. Set-valued terminals [4] extend the DDP terminal constraint to a data-inferred ellipsoidal set; our combination is complementary — physically driven explicit structure plus a per-term ablative decomposition.

C. Event-Driven Approaches and Contact-Time Uncertainty

The contact instant is a key variable of dynamic manipulation: prior work reparameterizes contact time and optimizes it explicitly [3], plans robustly over contact-timing uncertainty sets [15], and propagates gradients across contact events via the saltation matrix [16] — all requiring precise event detection and accurate models, or adopting hard robustness. Our candidate-window softmin is a *soft* aggregation instead: differentiable and free of event switching, it leaves the terminal target induced by the prediction window unfixed to a single candidate a priori; soft minima appear elsewhere in control mostly as constraint aggregations [17]. Across these families the axes of variation are the contact time (fixed, optimized, or set-valued), the spatial target (point, set, or tube), the guarantee (none or conditional), and hardware validation (frequent or simulation-only); ours occupies the candidate-induced, soft-corridor, empirical cell.

III. PROBLEM FORMULATION

A. Robot and Dynamics

We consider a dual-arm RM-65B manipulator on a vertical pole; the right arm carries a vertically mounted tennis racket

at the flange, the left arm is held at its zero configuration. The state collects the joint angles and velocities of both arms,

$$\mathbf{x} = [\mathbf{q}^\top, \dot{\mathbf{q}}^\top]^\top \in \mathbb{R}^{24}, \quad \mathbf{q} \in \mathbb{R}^{12},$$

and the control input is the right-arm joint torque $\mathbf{u} \in \mathbb{R}^6$ (torque mode). The discrete dynamics $\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)$ is provided by MuJoCo, and its analytic linearization (a batched C++ implementation) supplies the iLQR solver. The end-effector is the racket face center (the `racket_center` site).

B. Ball Prediction and Candidate Hit Windows

The ball is a standard tennis ball (radius 0.033 m, within ITF specifications). From ball position and velocity observations, the planning layer extrapolates a gravity-only parabola $\hat{\mathbf{p}}_b(t)$ (zero spin, no air drag; see limitations). The nominal hit time t_{hit} and hit point $\hat{\mathbf{p}}_{hit} = \hat{\mathbf{p}}_b(t_{hit})$ follow from intersecting that trajectory with the racket’s reachable region, and a *candidate hit window* $\mathcal{T} = \{t_{hit}^{(1)}, \dots, t_{hit}^{(K)}\}$ of half-width 50 ms is built around t_{hit} , each candidate carrying a desired triple of end-effector position, velocity, and normal along the trajectory.

C. Definitions

Definition 1 (Successful hit): Treating the ball and the racket as a geometry pair, a *contact* occurs if they form a narrow-phase collision pair within an episode. With racket face speed v_r at contact, $v_r > 0.3$ m/s yields an *active hit* and $0 < v_r \leq 0.3$ m/s a *passive contact* (the ball blocks against a nearly stationary racket). The metric throughout this paper is the *active-hit rate* over valid runs; passive contacts are counted separately and never enter it (below 2% of valid runs in every reported cell). A miss is the absence of any contact over the episode.

D. Constraints

Four-fold joint limits (position, velocity, acceleration, torque), a nominal TCP speed cap (1.8 m/s by default, 1.0 m/s for the constraint set inspired by the real robot’s limits; enforced at every step, including the hit neighbourhood), and a body-midline plane wall ($x \geq -0.1$ m) are enforced stage by stage through a multi-layer safety filter (Section IV-D).

IV. METHOD

A. Overall Framework

With the state $\mathbf{x} \in \mathbb{R}^{24}$ and control $\mathbf{u} = \boldsymbol{\tau} \in \mathbb{R}^6$ of Section III, the discrete dynamics $\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)$, $k = 0, \dots, N-1$, is linearized analytically ($\mathbf{A}_k, \mathbf{B}_k$; batched C++ implementation), and the iLQR solver computes gains ($\mathbf{K}_k, \mathbf{k}_k$) by a backward Riccati recursion and updates the trajectory by a forward line search with Levenberg–Marquardt regularization.

The cost (Figure 2) starts from the point terminal (1) and is modified by two independently switchable mechanisms (5): the running cost can gain the *spatial-corridor term* ℓ_k^{corr} (3),

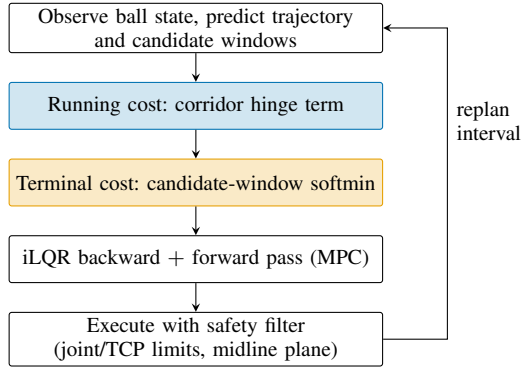


Fig. 2. MPC-iLQR framework. Shaded blocks are the two cost modifications; the point-target baseline (`none`) drops both.

and the terminal cost can be the *candidate-window softmin* ℓ_N^{sm} (4):

$$\ell_N^{pt}(\mathbf{x}_N) = \frac{1}{2} \|\mathbf{p}_{ee} - \mathbf{p}_{hit}\|_{\mathbf{Q}_p}^2 + \frac{1}{2} \|\mathbf{v}_{ee} - \mathbf{v}_{hit}\|_{\mathbf{Q}_v}^2 + \frac{1}{2} Q_n \|\mathbf{n}_{rack} - \mathbf{n}_{des}\|^2. \quad (1)$$

(1) penalizes the racket face center for deviating from a single desired hit point $\mathbf{p}_{hit}$, velocity $\mathbf{v}_{hit}$, and normal $\mathbf{n}_{des}$; all mechanisms below are stated relative to it.

B. Spatial-Corridor Running Cost

The corridor adds a lateral-tolerance running cost: it does not replace the terminal target — which remains the softmin (4) in full and the point terminal (1) in corridor-only — but shapes the approach so that the racket stays close to the predicted ball trajectory line within the candidate window. Let $\mathbf{p}_{ball,ref}$ be the ball position at the best candidate step (the corridor axis passes through it) and $\hat{\mathbf{d}}_{ball} = \text{normalize}(\sum_k \bar{w}_k \mathbf{v}_{ball,k})$ the window-averaged trajectory-line direction (weights $\bar{w}_k$ as defined in (4)). The perpendicular projection matrix is

$$\mathbf{P}_\perp = \mathbf{I} - \hat{\mathbf{d}}_{ball} \hat{\mathbf{d}}_{ball}^\top, \quad (2)$$

and the corridor running cost is a hinge loss,

$$\ell_k^{corr}(\mathbf{x}_k) = \frac{1}{2} s_k q_{corr} \left[\max(0, \|\mathbf{P}_\perp(\mathbf{p}_{ee,k} - \mathbf{p}_{ball,ref})\| - r_{racket}) \right]^2, \quad (3)$$

where $r_{racket} = 0.12$ m is the racket radius (the corridor half-width), q_{corr} a scalar weight, s_k a per-step weight scaling (both instantiated below), $\hat{\mathbf{d}}_{ball}$ a window-averaged trajectory direction, and ℓ_k^{corr} is applied only to the steps k inside the candidate window. Only the *perpendicular* deviation is penalized — leading or lagging along the line (timing error) is free, so the solver does not commit to a single time-space correspondence — and inside the band the cost is zero, encoding a contact tolerance of one racket radius around the ball center (Figure 3). As a geometric surrogate, (3) asserts no contact within the band and is the only corridor running term: velocity-direction and normal alignment are carried by the softmin terminal, and r_{racket} takes the nominal 0.12 m (sensitivity in Section VI).

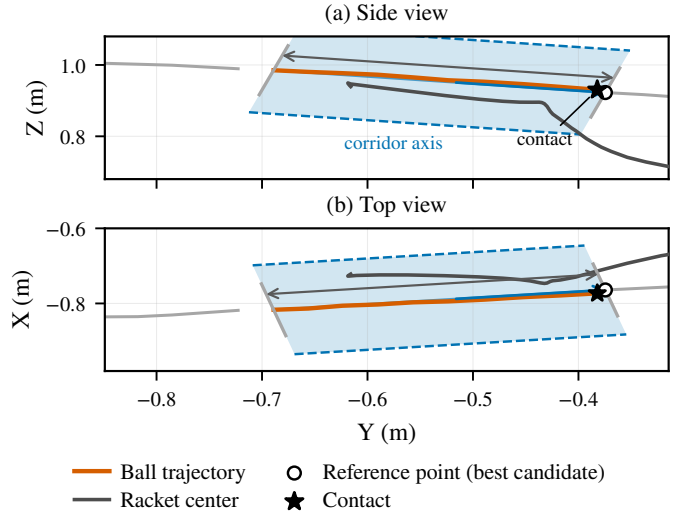


Fig. 3. Spatial corridor along the predicted ball trajectory, zoomed on the candidate window, drawn here as the 50 ms of the trajectory ending at the best candidate step (the full flight is in Figure 1): (a) side and (b) top view of one swing, sharing the forward-axis range. The corridor is a ± 0.12 m band around a straight-line approximation of the ball line within the window (orange: window segment of the trajectory; gray: outside it); the circle is the reference point at the best candidate step, the star the actual contact.

C. Candidate-Window Terminal

The point target also forces “when” to hit to be unique. We center a candidate window $\mathcal{W} = \{k : |t_k - t_{best}| \leq 50 \text{ ms}\}$ at the predicted hit step k_{best} , form a candidate cost c_i for each $i \in \mathcal{W}$ in the form of (1), and aggregate with a softmin:

$$\ell_N^{sm}(\mathbf{x}_N) = -\frac{1}{\beta} \log \sum_{i \in \mathcal{W}} \bar{w}_i \exp(-\beta c_i),$$

$$\tilde{w}_i = \exp[-(t_i - t_{best})^2 / (2\sigma^2)], \quad (4)$$

$$\bar{w}_i = \frac{\tilde{w}_i}{\sum_{j \in \mathcal{W}} \tilde{w}_j}, \quad \beta = 5.0,$$

with $\sigma = 50$ ms (the normalization contributes only a state-independent constant; the weights favor candidates near the nominal prediction and are not a calibrated probability model of prediction error). Every candidate cost c_i is evaluated at the *same* terminal state $\mathbf{x}_N$ against its own desired triple $(\mathbf{p}_i, \mathbf{v}_i, \mathbf{n}_i)$ along the predicted trajectory: no cross-time state coupling is introduced, so the softmin represents a *target set induced by timing uncertainty* rather than a contact-time selection mechanism within a single solve. Replanning refreshes the prediction and target set between solves. Each candidate cost is quadratic in a task-space residual, $c_i(\mathbf{x}) = \frac{1}{2} \mathbf{r}_i^\top \mathbf{W}_i \mathbf{r}_i$ with $\mathbf{r}_i = (\mathbf{p}_{ee} - \mathbf{p}_i, \mathbf{v}_{ee} - \mathbf{v}_i, \mathbf{n}_{rack} - \mathbf{n}_i)$ and $\mathbf{W}_i \succeq 0$. ℓ_N^{sm} enters the backward recursion as an ordinary terminal cost, evaluated with the Gauss–Newton Hessian $\sum_i \alpha_i \mathbf{J}_i^\top \mathbf{W}_i \mathbf{J}_i$, where $\mathbf{J}_i = \partial \mathbf{r}_i / \partial \mathbf{x}$ and $\alpha_i = \bar{w}_i e^{-\beta c_i} / \sum_j \bar{w}_j e^{-\beta c_j}$: a weighted sum of positive-semidefinite terms keeps the Riccati recursion well posed in practice, further stabilized by Levenberg–Marquardt damping. The exact Hessian additionally contains residual-curvature terms and the negative covariance term induced by the softmin; both are omitted in our positive-semidefinite Gauss–Newton approximation. As $\beta \rightarrow \infty$ the aggregation

becomes a hard min, dominated by the best candidate at the nominal weights, and empirically a hard-min terminal ($\beta = 10^6$) is indistinguishable from the softmin (Section VI). The operative mechanism is therefore the *candidate target set* rather than the smoothness of the aggregation: candidates induced by timing uncertainty provide alternative terminal targets, so a prediction shift that moves the best candidate between replans does not change the problem structure, and the aggregation embeds directly into the backward recursion. The two terms are independently switchable ((5)) and are *designed* to tolerate spatial and temporal prediction errors, respectively; Section VI measures the conditions under which each design intent pays off instead of assuming it.

Reproduction details. At the 5 ms step the ± 50 ms window spans 21 slots; workspace, height, and IK-margin filters retain 2–21 (typically ≈ 13). The corridor scaling s_k equals the normalized weight $\bar{w}_k$; $q_{\text{corr}} = 0.01 q_p = 5 \times 10^2$ with $Q_p = q_p I$, $q_p = 5 \times 10^4$; $\beta = 5.0$.

D. MPC Integration and Safety Filtering

Replanning. Replanning occurs every $T_{re} = 30$ steps (150 ms at 200 Hz), scheduled by the steps remaining to the hit: far ($k_{hit} > 50$) uses the Jacobian-transpose guess $\mathbf{u} \propto \mathbf{J}_p^\top(\mathbf{p}_{hit} - \mathbf{p}_{ee})$ with zero iLQR iterations, while mid and near ($k_{hit} \leq 50$) add iterations and hard-constraint checks. The control penalty $\mathbf{R}$ anneals as the hit approaches, with an extra decay on joint 1 for a shoulder-driven swing.

Safety filtering. Each step passes three checks: four-fold joint limits, a nominal TCP speed cap v_{tcp}^{max} , and the body-midline plane wall ($x \geq -0.1$ m, pushed back by PD when violated). The cap is enforced at every step, including the hit neighbourhood: a fallback gain sequence $\alpha_s \in \{1.0, 0.8, 0.6, 0.4, 0.2, 0.0\}$ scales the candidate control, and the largest value whose *one-step prediction* passes every check is kept (scaling, not clipping). The realized peak speed can overshoot the cap transiently (Section VI); if every scaling fails, a damping brake ($\mathbf{u} = -20 \dot{\mathbf{q}}$) applies as an emergency fallback. The flag it sets marks a fallback, not a checked-safe control; in the recorded real-time runs it fired in 3 of 40 episodes (two asynchronous, one synchronous).

Follow-through. For 60 steps (300 ms) after contact the end-effector keeps accelerating opposite to the incoming ball (length 0.5 m), preventing deceleration from shifting the hit point.

Real-time budget. Upon serve detection the controller performs one synchronous 30-iteration first plan (≈ 0.93 s on the test CPU) before motion begins, simulated time frozen; steady-state replanning thereafter fits the 150 ms budget, and asynchronous execution prevents solver latency from blocking the control loop ($1.0\% \rightarrow 0.04\%$ of steps beyond the 5 ms period; Section VI).

E. Point-Target Baseline (Degenerate Case)

(1) prescribes a one-to-one time–space correspondence by penalizing, at a single predicted instant t_{hit} , deviations from one predicted target: a prediction error δt or $\delta \mathbf{p}$ shifts that

prescribed target away from the true contact state; Section VI quantifies the asymmetry of the resulting cost.

One family, two switches. With ℓ_k^{base} the control-effort and smoothness running cost, the family reads

$$J_{a,b} = \sum_k (\ell_k^{base} + a \ell_k^{corr}) + (1-b) \ell_N^{pt} + b \ell_N^{sm}, \quad a, b \in \{0, 1\}, \quad (5)$$

so the corridor *adds* a running term while the softmin *replaces* the point terminal. The four ablation tiers are full $(a, b) = (1, 1)$, corridor-only $(1, 0)$, candidate-set-only $(0, 1)$, and point-target $(0, 0)$. Apart from these two switches all tiers share identical weights, warm-start, initialization, and safety rules; the sensitivity sweep over β , the window half-width, and r_{racket} is in Section VI.

V. EXPERIMENTS

Common protocol. All experiments run the dual-arm RM-65B in MuJoCo (right arm, 6 driven joints; left arm zeroed) at a 5 ms step, with serves drawn from a randomized serve box (ball speed 7–15 m/s). The metric is the active-hit rate of Definition 1 (Section III-C): a ball–racket collision pair with racket face speed > 0.3 m/s. For all multi-tier comparisons, seeds $(1 \dots N)$ are paired across tiers, isolating the algorithmic configuration under otherwise identical conditions. Perturbations are injected on the planning side at each replan. The temporal perturbation shifts the predicted hit step, moving the horizon, the iteration-phase schedule, and the control-penalty annealing identically across tiers. The spatial perturbation is a *hit-point* perturbation: an offset perpendicular to the predicted trajectory (horizontal plane) applied to the predicted hit point, which shifts the point-target terminal and, in every tier, the Jacobian-transpose warm start, whereas the candidate window and the corridor axis are rebuilt from the current ball observation at every replan and do not carry it. It therefore measures one specific failure mode — a biased single point target while the trajectory geometry is still available — not a uniformly biased prediction; shared seeds do not remove this difference in what each tier receives, and the claims below are restricted accordingly (Section VI reports a shared-input variant). Both magnitudes are uniform in $[0, \max]$ with a random, seeded sign and decay with the replan index as $\max(\alpha_{\min}, 1 - r_{count}/r_{total})$ (reaching ≈ 0.44 of the maximum at the final pre-hit replan), emulating a prediction that improves as the ball approaches. Runs without a verdict (no result line) are excluded (0.9–3.0% of rows, evenly distributed across tiers). Because the design is paired, all comparisons use the *exact paired McNemar test* on same-seed outcomes with paired-bootstrap 95% confidence intervals; the limit $\times$ mechanism interaction uses a paired difference-in-differences (per-seed gain difference); grid heatmaps are corrected by the Holm procedure over the 20 cells.

A. Capability Sweep and the Real-Robot Constraint Set

The speed sweep (9 speeds $\times$ 200 seeds, default TCP limit 1.8 m/s) maps the task’s capability envelope; the constraint-set

TABLE I

GAIN (PP) OVER THE POINT-TARGET BASELINE UNDER THE TWO TCP CAPS (7 M/S, $n = 394$ PAIRED SEEDS); PAIRED MCNEMAR p ; DiD = PAIRED PER-SEED GAIN DIFFERENCE (BOOTSTRAP 95% CI; SIGN TEST ON DIRECTION); p -VALUES UNADJUSTED, PRE-SPECIFIED COMPARISONS.

Config.	Gain over point-target (pp)		p	
	TCP 1.8	TCP 1.0	TCP 1.8	TCP 1.0
corridor-only	+3.3	+5.8	0.171	0.027
candidate-set-only	+1.3	-11.9	0.668	< 0.001
full	+1.5	-12.7	0.594	< 0.001
Paired DiD (gain shift from TCP 1.8 to TCP 1.0):				
corridor-only	+2.5 pp (CI [-4.1, 9.1]; sign test $p = 0.466$)			
candidate-set-only	-13.2 pp (CI [-19.8, -6.6]; sign test $p < 0.001$)			
full	-14.2 pp (CI [-20.8, -7.6]; sign test $p < 0.001$)			

comparison (7 m/s $\times$ 200 seeds, the same seeds under both limit layers, the tight one a nominal 1.0 m/s cap enforced at every step as in Section IV-D) prices that set. A representative swing and the two conditions are shown in Figure 5 and Figure 6.

B. Four-Tier Ablation and the Perturbation Grid

We refer to the four ablation flags `full`, `tube_only`, `softmin_only`, `none` as `full`, `corridor-only`, `candidate-set-only` (softmin implementation), and `point-target`.

Nominal four-tier ($n \approx 98$ per cell) runs all tiers at 7/9/12 m/s without perturbation. **Perturbation grid** (9 m/s, replicated at 7 m/s) crosses time maxima $t \in \{0, 10, 25, 50, 100\}$ ms with spatial maxima $s \in \{0, 0.05, 0.1, 0.2\}$ m, 20 cells per tier ($n \approx 1960$). **Pre-specified high-power corners** (9 and 12 m/s, $s \in \{0.2, 0.3, 0.4\}$ m at $t = 0$, plus $t = 50$ ms at $s = 0.2$ m, $n = 1481$ –2986 per cell) targets the corridor’s *independent* value with sufficient power; comparisons were fixed before the runs. Per-condition rates of the four tiers are collected in Table II; results are reported per research question in Section VI.

C. Limit $\times$ Mechanism Pairing

The limit pairing (7 m/s, $2 \times 4 \times 400$ seeds, same-seed aligned across both limit layers and all tiers) crosses the two TCP layers with the four tiers to test whether the mechanism ranking depends on the speed budget (Table I); representative same-seed diagnostics in the anonymized artifact sanity-check the hit/miss separation (Section VI).

D. Real-Time Budget

The real-time study runs 30 synchronous and 10 asynchronous episodes at 9 m/s on an idle machine (single workstation, one solver thread; Intel i9-12900K), logging every replan duration and per-step latency (Figure 7).

VI. RESULTS AND DISCUSSION

A. RQ0: Capability Envelope and the Cost of the TCP Constraint Set

Under default limits the active-hit rate over $7 \rightarrow 15$ m/s reads 84.3/94.4/94.4/87.8/83.3/84.8/64.8/73.4/58.0%: it peaks at 8–9 m/s ($\sim 94\%$) and falls to 58.0% at 15 m/s

(Wilson 95% CI half-width 3.3–6.8 pp, $n \approx 198$). The robot-derived constraint set (nominal TCP cap 1.0 m/s, enforced at every step) at 7 m/s lowers the rate from 84.4 to 35.9% (–48.7 pp, paired CI [-56.9, -40.6], $p < 0.001$) and compresses the mean racket speed on hit rows from 1.50 to 0.88 m/s: the cap acts as an execution-time limit rather than a planning guideline. We call these settings *speed-rich* and *speed-scarce*.

B. RQ1: What Does the Candidate-Window Terminal Contribute?

Under nominal conditions the candidate window captures nearly the entire gain over the point target: +17.4 pp at 9 m/s (CI [+9.2, +25.5]) and +32.7 pp at 12 m/s (CI [+22.4, +42.9]), with `full` = `candidate-set-only` there (0.0 pp, $p = 1.0$; Figure 4(a)). At 7 m/s the gain is +2.1 pp and no pairwise difference is detected (≤ 5.2 pp, all $p \geq 0.40$); the TCP 1.8 layer of the limit pairing agrees (Table I). On the merged 20-cell grid `candidate-set-only` reaches 82.8% against 73.9% for the point target (+8.9 pp, CI [+6.9, +11.0]), while `full` and `candidate-set-only` differ by 0.3 pp ($p = 0.06$). After Holm adjustment only the two zero-spatial-error cells are individually significant (Figure 4(b)); a per-seed interaction $I(t, s) = [H_{sm}(t, s) - H_{pt}(t, s)] - [H_{sm}(0, 0) - H_{pt}(0, 0)]$ is negative in all 19 perturbed cells ($I(0, 0) = 0$ by construction), from -2.0 to -19.4 pp (mean -8.9 pp): the advantage is largest unperturbed and *narrows* under both perturbation types, so the nominal gain partially persists under both perturbation types rather than splitting by mechanism.

C. RQ2: When Does the Corridor Provide Independent Value?

At the pre-specified corners, the corridor’s independent value (corridor-only vs. point-target; same biased point target, only the former adds the corridor) is +4.4 pp (9 m/s, $t=0$, $s=0.2$ m; CI [+2.8, +6.1], $p < 0.001$, $n = 2970$) and +6.9 pp (12 m/s, same corner; CI [+5.4, +8.4], $p < 0.001$, $n = 2986$): the point estimate is larger at 12 m/s, consistent with a tighter time-margin interpretation, although speed-specific factors are not separately controlled. At 9 m/s the point estimate remains positive at $s = 0.3$ m (+2.0 pp, $p = 0.081$, n.s.) and is +2.6 pp at $s = 0.4$ m (CI [+0.5, +4.7], raw $p = 0.016$). The largest value occurs at the smallest swept magnitude, $s = 0.2$ m; the interior maximum is not located. Adding a 50 ms temporal perturbation to the same corner leaves only +2.0 pp ($p = 0.115$). In the full system this independent value is covered by the candidate window: at the corners `full` and `candidate-set-only` differ by ± 0.0 pp ($p = 1.0$), and on the grid no cell of `full` – `candidate-set-only` is significant even without correction (Figure 4(c)) – an absence of a detected effect, not an equivalence claim. Representative same-seed pairs (in the anonymized artifact) show the same separation: in each pair the hit run approaches the 0.12 m racket radius, the miss stays above it.

Shared-input check. Above, only the point target carries the spatial bias, leaving the candidate set and the corridor axis with an unbiased reference. Repeating the spatially perturbed cells with the bias applied to the ball state consumed by the

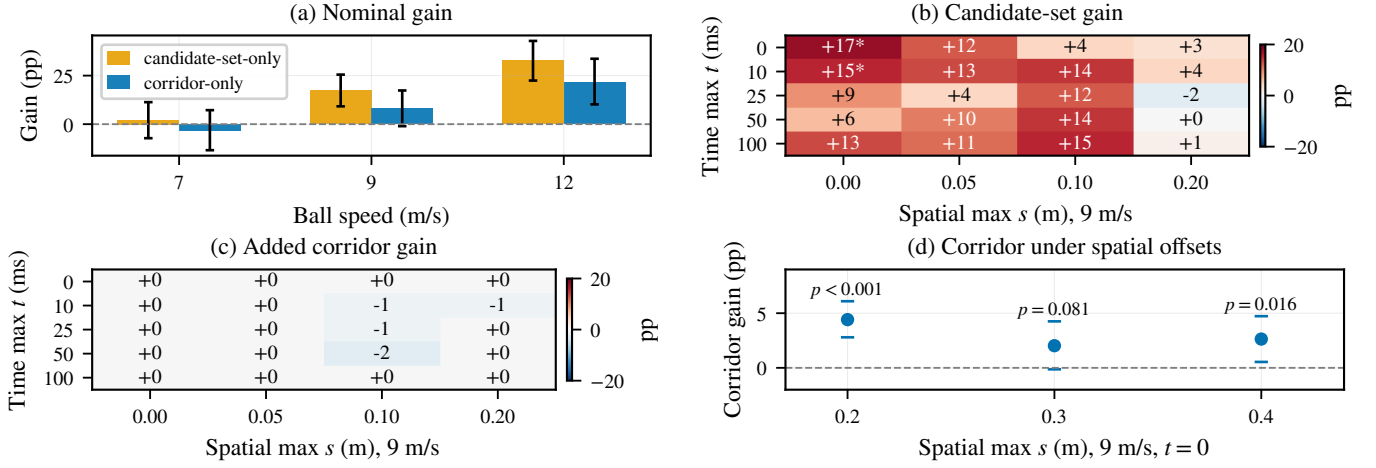


Fig. 4. Mechanism attribution: the candidate window carries nearly the whole nominal gain, *corridor-only* is significant under selected spatial perturbations, and no additional corridor effect appears once the window is active. (a) Nominal gains over the point target vs. ball speed (paired-bootstrap 95% CI). (b) *candidate-set-only* gain and (c) added corridor gain (full – *candidate-set-only*) on the 20-cell perturbation grid at 9 m/s, shared ± 20 pp scale (all 20 cells of (c) within 2 pp); stars mark cells significant after Holm adjustment. (d) *corridor-only* value vs. perturbation magnitude at 9 m/s, $t=0$ (pre-specified high-power corners, $n = 1481\text{--}2970$; p-values uncorrected). The spatial perturbation biases the point-target hit point only; the shared-input variant (Section VI) biases the ball state consumed by the planner instead.

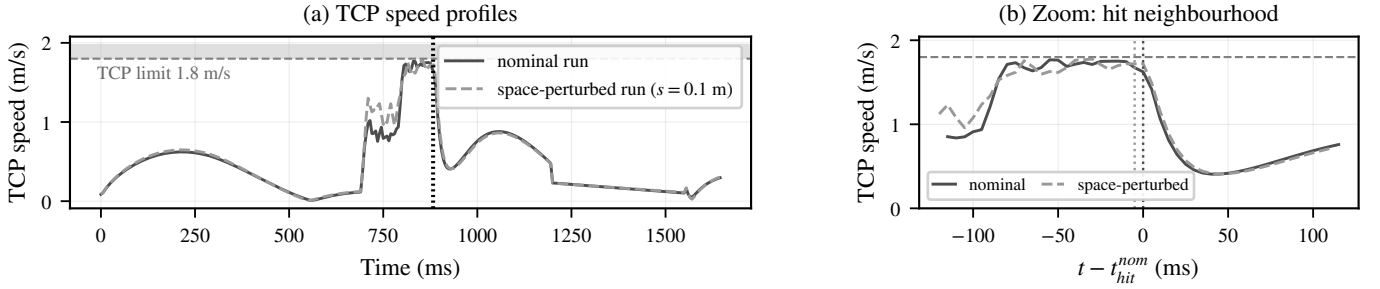


Fig. 5. Representative swing under the default limits: (a) TCP speed profiles of a nominal and a space-perturbed run ($s = 0.1$ m) against the 1.8 m/s cap, the joint excursion staying within 46% of the available limit headroom; (b) zoom on the hit neighbourhood: the runs differ by up to $\Delta v_{max} = 0.38$ m/s and by -5 ms at contact (dotted lines matched to each run’s line style, axis relative to the nominal contact).

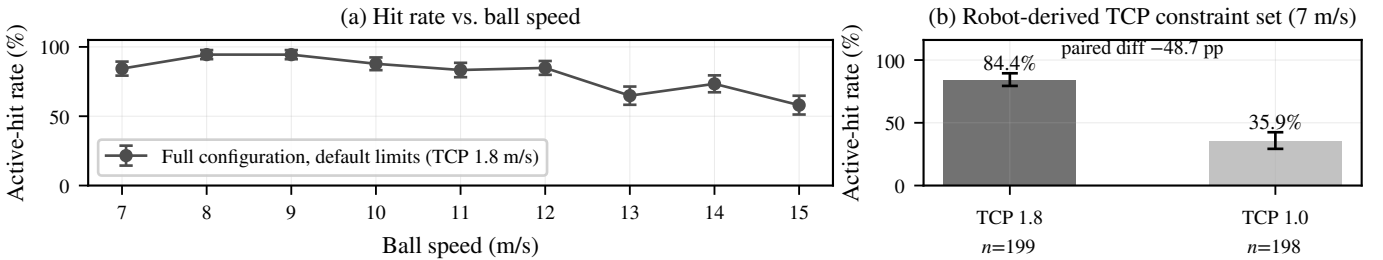


Fig. 6. Capability and the cost of the TCP constraint: (a) active-hit rate vs. ball speed under default limits; (b) default vs. the 1.0 m/s constraint set at 7 m/s; Wilson 95% intervals and per-condition n , with the paired difference over the $n=197$ same-seed pairs.

planner (51,000 more episodes; timing and reachability still use the current observation) leaves every mechanism value essentially unchanged (Table III): the reported values hold under the tested shared-input bias condition.

Sensitivity of the nominal parameters. The nominal choices are not knife-edge. At 9 m/s (200 seeds per setting), the full system stays within 93.9–94.9% across $\beta \in \{1, 10\}$, half-widths $\{25, 75\}$ ms, and radii $\{0.08, 0.16\}$ m (nominal 95.9%), candidate-set-only within 93.9–94.9% (paired $p < 0.001$), and corridor-only at 84.8% for both radii (nominal 86.7%; $p = 0.18$). A numerical hard-min over the same

candidate set ($\beta = 10^6$, 200 seeds: 187/197 for full, 186/197 for candidate-set-only) is consistent with the observed gain arising primarily from the candidate set rather than soft aggregation: on the 98 seeds valid in both runs, hard-min and the nominal softmin ($\beta = 5$) reach the same 94/98 hits with zero discordant pairs — the apparent 94.9% vs. 95.9% gap is only the different sample sizes ($n = 197$ vs. $n = 98$).

D. RQ3: How Does the TCP Constraint Change the Ranking?

The limit $\times$ mechanism pairing changes the tier ranking (Table I). Under TCP 1.8 no tier separates from the point

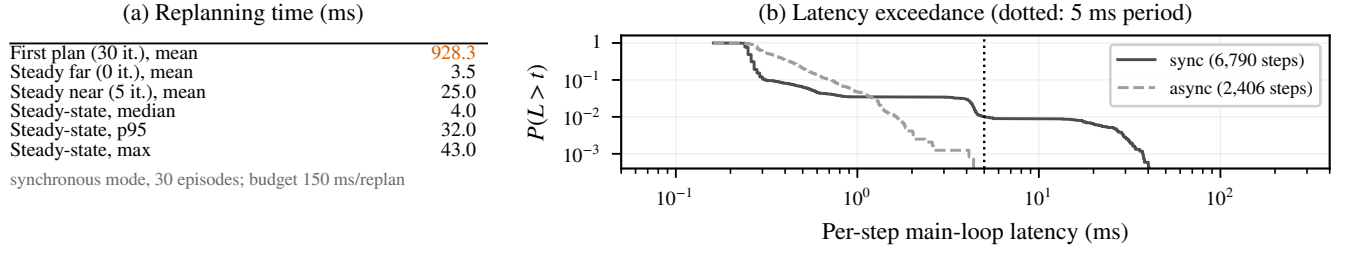


Fig. 7. Real-time budget after initialization: steady-state replanning meets the 150 ms budget with ample margin, and the per-step latency tail of the synchronous mode is what causes main-loop stalls. (a) Replanning time per phase (numeric summary, synchronous mode; the first plan consumes the serve’s prediction before any motion). (b) Exceedance probability $P(L > t)$ of the per-step main-loop latency over all recorded steps, log axes, so the tail is visible directly: the synchronous mode exceeds the 5 ms control period in 68/6790 steps (1.0%) and the asynchronous mode in 1/2406 (0.04%).

TABLE II

ACTIVE-HIT RATE (%) OF THE FOUR CONFIGURATIONS ACROSS CONDITIONS; n = VALID RUNS PER CONFIGURATION, GIVEN AS THE RANGE ACROSS THE FOUR (THE PERTURBATION-GRID ROW POOLS ITS 20 CELLS); EXACT INTERVALS ARE IN THE ARTIFACT.

Condition	full	corridor-only	candidate-set-only	point-target	n	Design
Nominal 7 m/s	83.8	78.4	83.5	81.4	97–99	nominal four-tier
Nominal 9 m/s	95.9	86.7	95.9	78.6	98	nominal four-tier
Nominal 12 m/s	85.7	74.5	85.7	53.1	98	nominal four-tier
Perturbed grid (20 cells, 9 m/s)	82.6	81.2	82.8	73.9	1960–1962	perturbation grid
Corner 12 m/s, $t=0$, $s=0.2$ m	64.8	61.6	64.8	54.7	2986–2987	high-power corners

TABLE III

SHARED-INPUT CHECK (51,000 EPISODES): PAIRED GAINS (PP) OVER THE POINT TARGET UNDER THE ORIGINAL HITPOINT BIAS AND THE SHARED-INPUT BALL-STATE BIAS; BOOTSTRAP 95% CI IN BRACKETS, n PAIRED ($p < 0.001$ EXCEPT THE LAST ROW: $p=0.06$ HITPOINT, $p=1.0$ BALL-STATE, ZERO DISCORDANT PAIRS).

	hitpoint bias	ball-state bias
corridor-only – point-target, 9 m/s corner	+4.4 [+2.8, +6.1], $n=2970$	+4.3 [+2.7, +6.0], $n=2969$
corridor-only – point-target, 12 m/s corner	+6.9 [+5.4, +8.4], $n=2986$	+7.0 [+5.5, +8.5], $n=2987$
candidate-set-only – point-target, grid	+8.9 [+6.9, +11.0], $n=1960$	+10.2 [+8.1, +12.2], $n=1960$
full – candidate-set-only, grid	−0.3 [−0.5, −0.1], $n=1960$	0.0, $n=1960$

target (all $p \geq 0.17$); under the nominal 1.0 m/s cap the ordering changes: candidate-set-only loses −11.9 pp and full −12.7 pp (paired $p < 0.001$ for both; mean paired DiD −13.2 pp, bootstrap CI [−19.8, −6.6], and −14.2 pp, CI [−20.8, −7.6], sign test $p = 1.2 \times 10^{-4}$, $p < 0.001$), whereas corridor-only gains +5.8 pp (CI [+1.0, +10.7], $p = 0.027$) and achieves the highest observed rate under the limit. On the $n = 93$ seeds where both tiers hit, the point target attains the smallest mean hit-time error (14.4 ms) at the slowest swing (0.76 m/s), whereas the softmin attains 17.2 ms at 0.87 m/s — a pattern consistent with the softmin spending part of its speed budget hedging across candidates, though this subset excludes the missed runs. A numerical hard-min under the same cap is consistent with that interpretation: the rates move by at most 0.3 pp (full 123 vs. 124/394, candidate-set-only 126 vs. 127/394; −0.3 pp, CI [−0.8, 0.0], one discordant pair each) while both stay 12–13 pp below the point target (44.2%); under these conditions the hard-min does not remove the decline, so the smoothness of the aggregation alone does not explain it. Peak TCP speeds exceed the cap in 19/8/19/6% of episodes (full/corridor-only/candidate-set-only/point-target): most excursions are marginal and transient ($p90 \leq 1.01$ m/s), and the 13–14 per tier that reach larger peaks (> 1.2 m/s) are all misses with tier-identical peak speed and joint velocity at the same seed — a shared unstable initial

condition, not a mechanism difference. Hedging presupposes speed headroom, so under the cap it would become detrimental — a possible explanation consistent with these observations, confined to the tested condition (7 m/s, 1.0 m/s cap); we do not extrapolate.

E. RQ4: Does Replanning Meet the Runtime Budget?

In 30 synchronous episodes (291 replans; plus 94 in 10 asynchronous ones), steady-state replanning takes 3.5 ms (far) and 25.0 ms (near): 4 ms median, 32 ms p95, 43 ms maximum, inside the 150 ms budget with ample margin. The bottleneck is the first plan of each episode, a synchronous 30-iteration solve (928.3 ms over the $n = 28$ first solves) that consumes the serve’s prediction before motion starts with simulated time frozen: the steady-state budget describes the loop *after* initialization, not the response to a mid-run serve (idle Intel i9-12900K). What matters for control is the tail (Figure 7(b)): the synchronous mode exceeds the 5 ms control period on 1.0% of steps (deepest 43.3 ms), the asynchronous mode on 0.04% (one step, deepest 5.3 ms) — asynchronous replanning buys non-blocking execution, not faster solving.

F. Discussion

Positioning and implication. Relative to the “point-target + high-bandwidth perception” route [8]–[10] this work is complementary — where the sensing budget is tighter, cost design

compensates — and relative to event-time reparameterization [3] and robust contact-timing optimization [15] it delimits the *regime of applicability* of each layer. A robustness layer’s value can change sign, so mapping its regime should be standard practice in cost design.

Limitations. The study is simulation-based, isolating the algorithmic factors; no hardware validation is claimed. Prediction error is injected as a hit-point position offset and a timing shift, approximating several prediction-error sources (including unmodeled drag); the spatial side biases the predicted target geometry rather than the timing, and the shared-input variant leaves the corridor value within 0.1 pp; zero spin and no air drag are modeled. The nominal weights are one configuration, and the one-at-a-time sweep over β , the window half-width, and the corridor radius (Section VI) makes the reported boundaries conditional on them. Evidence covers one task and one arm type, and the reversal rests on a single condition (7 m/s $\times$ TCP 1.0). Active contact is the task metric; post-impact return quality (outgoing velocity, net clearance, landing accuracy) is not evaluated. Accounting: 114,810 episodes were simulated across the reported conditions (the perception sweeps are in the anonymized artifact); 1.3% terminated without a verdict, and the exclusion share differs by at most 0.5 pp across tiers within every experiment (e.g. 12/800 per tier in the limit pairing). Every rate, interval, and test statistic here is regenerated by one analysis script from the per-episode logs.

VII. CONCLUSION

In our setting the point-target terminal is brittle to spatiotemporal prediction errors because it compresses a feasible time-space set into a single target. Starting from it in an MPC-iLQR framework, we studied a two-switch cost family — a *spatial corridor* added as a hinge position-deviation running cost along the predicted ball trajectory, and a *candidate-window softmin* that replaces the point terminal by a soft aggregation over candidate ball states — over 110k controlled episodes. The candidate window carries the dominant gain in the speed-rich regime; the smoothness of the aggregation alone cannot explain that gain; the corridor’s independent value appears only where time margins are tight and the error is spatial (a biased hit point; a shared-input variant leaves the corridor value within 0.1 pp); under a nominal 1.0 m/s TCP cap the candidate-window tier reverses to harmful while the corridor stays beneficial. For designers of dynamic manipulation, the takeaway is that the robustness of a cost

layer is task-condition-dependent and should be mapped, not assumed. Future work: real-robot deployment, continuous rallying, closed-loop perception, limit-aware mechanism switching, and extending the spatial protocol to a uniformly biased ball-state prediction.

REFERENCES

- [1] E. Todorov and W. Li, “A generalized iterative LQG method for locally-optimal feedback control of constrained nonlinear stochastic systems,” in *Proc. American Control Conference (ACC)*, Jun. 2005, pp. 300–306.
- [2] Y. Tassa, T. Erez, and E. Todorov, “Synthesis and stabilization of complex behaviors through online trajectory optimization,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, Oct. 2012, pp. 4906–4913.
- [3] T. Gossard, T. Köpf, and A. Ziegler, “Event-time hybrid optimal control for robotic table tennis serves,” 2026, arXiv:2608.08157.
- [4] S. Eom and G. Park, “Differential dynamic programming for the optimal control problem with an ellipsoidal target set and its statistical inference,” in *Proc. Int. Conf. Control, Automation and Systems (ICCAS)*, Oct. 2025, pp. 1960–1965.
- [5] T. Senoo, A. Namiki, and M. Ishikawa, “High-speed batting using a multi-jointed manipulator,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2004, pp. 1191–1196.
- [6] K. Mülling, J. Kober, and J. Peters, “A biomimetic approach to robot table tennis,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, Oct. 2010, pp. 1921–1926.
- [7] K. Mülling, J. Kober, O. Kroemer, and J. Peters, “Learning to select and generalize striking movements in robot table tennis,” *The International Journal of Robotics Research*, vol. 32, no. 3, pp. 263–279, 2013.
- [8] D. Nguyen, K. D. Cancio, and S. Kim, “High speed robotic table tennis swinging using lightweight hardware with model predictive control,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, May 2025, pp. 15 278–15 284.
- [9] Z. Su, B. Zhang, N. Rahmanian, Y. Gao, Q. Liao, C. Regan, K. Sreenath, and S. S. Sastry, “HITTER: A Humanoid Table Tennis robot via hierarchical planning and learning,” 2025, arXiv:2508.21043.
- [10] G. Torrente, G. J. Maeda, D. Grover, M. Tsukamoto, H. Sahloul, and P. Dürr, “Ace! motion planning of professional-level table tennis serves with a robot arm,” 2026, arXiv:2607.06989.
- [11] S. Kim, A. Shukla, and A. Billard, “Catching objects in flight,” *IEEE Transactions on Robotics*, vol. 30, no. 5, pp. 1129–1145, 2014.
- [12] Y. Tassa, N. Mansard, and E. Todorov, “Control-limited differential dynamic programming,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2014, pp. 1168–1175.
- [13] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert, “Constrained model predictive control: Stability and optimality,” *Automatica*, vol. 36, no. 6, pp. 789–814, 2000.
- [14] Y. Liu and A. Billard, “Tube acceleration: Robust dexterous throwing against release uncertainty,” *IEEE Transactions on Robotics*, vol. 40, pp. 2831–2849, 2024.
- [15] H. Zhao and M. Khadiv, “Trajectory optimization under contact timing uncertainties,” 2024, arXiv:2407.11478.
- [16] N. J. Kong, G. Council, and A. M. Johnson, “iLQR for piecewise-smooth hybrid dynamical systems,” in *Proc. IEEE Conf. Decision and Control (CDC)*, Dec. 2021, pp. 5374–5381.
- [17] P. Rabiee and J. B. Hoagg, “Soft-minimum barrier functions for safety-critical control subject to actuation constraints,” in *Proc. American Control Conference (ACC)*, 2023, pp. 2646–2651.